

```
In [ ]: ▶ # n1 = int(input()) # 10
# n2 = int(input()) # 20
# print(n1+n2)
# print (*range(2,10,2))
print (*range(2,10))
```

```
In [ ]: ▶ def isPrime(n):
    for fact in range(2,n):
        if n % fact == 0 :
            return False
    return True

# main function
n = int(input())
print ("Prime" if isPrime(n) else "Not Prime")
```

```
In [ ]: ▶ #Program to check whether the given is prime or not
def isPrime(n):
    for fact in range(2,n//2):
        if n % fact == 0 :
            return False
    return True

# main function
n = int(input())
print ("Prime" if isPrime(n) else "Not Prime")
```

```
In [ ]: ▶ #Program to check whether the given is prime or not
import math as m
def isPrime(n):
    for fact in range(2,int(m.sqrt(n))+1):
        if n % fact == 0 :
            return False
    return True

# main function
n = int(input())
print ("Prime" if isPrime(n) else "Not Prime")
```

```
In [ ]: """
All prime numbers are either 6N + 1 or 6N -1 except 2 & 3
but
All 6N+1 and 6N-1 are not prime numbers

"""
import math as m
def isPrime(n):
    if n == 1:
        return False
    if n > 3 and (n % 2 == 0 or n % 3 == 0):
        return False
    for fact in range(5,int(m.sqrt(n))+1,6):
        if n % fact == 0 or n % (fact+2)==0:
            return False
    return True

# main function
n = int(input())
print ("Prime" if isPrime(n) else "Not Prime")
```

```
In [ ]: print(*range(5,101, 6))
```

```
In [ ]: ## for index in range(n):
# #     data[index] = int(data[index])
# print (data[0]+data[1])
# print(data)

# list1 = [10,21,33,4,5,6]
# slicing
# [startIndex:endIndex:incrementValue]
# print(list1[:3]) # print first 3 values. It prints oth, 1st, 2nd index v
# print(list1[3:])
# print (list1[::-1])
# print (list1[3::-1])
```

```
In [ ]: ▶ #Program to sort the list using selection sort algorithm - greedy
def selectionSort(dataList, n):
    for i in range(n):
        mi = i
        # selection least value from the remaining list
        for j in range(i+1, n):
            if dataList[mi] > dataList[j]:
                mi = j
        #checking whether ith is least or not
        if i!=mi:
            dataList[mi], dataList[i] = dataList[i],dataList[mi]

n = int(input())
# data = list(map(int, input().split()))[:n]
data = []
for _ in range(n):
    data.append(int(input()))
selectionSort(data, n)
print(data)
```

```
In [ ]: ▶ list1 = ["Jegan", "Dhanya", "Nathish", "Yuvasri", "Sudharsan", "Isaimani",
# list1.sort(reverse=True) #descending order
# sortedList=sorted(list1)
sortedList=sorted(list1, reverse=True) #descending order
print(list1)
print(sortedList)
```

```
In [ ]: ▶ # Day - 2
#Program to print natural numbers using iterative method
n = int(input())
# for i in range(1,n+1):
#     print(i,end=" ")
print(*range(1,n+1))
```

```
In [ ]: ▶ #Program to print natural numbers using recursive method
def printNatural(x):
    if x < 1:
        return
    print(x, end=" ")
    # recursive call
    printNatural(x-1)

# main function
n = int(input())
printNatural(n)
```

```
In [ ]: ▶ # def printNatural(5):  
#         if 5 < 1:  
#             return  
#         printNatural(5-1)  
#         print(5, end=" ")  
  
# def printNatural(4):  
#         if 4 < 1:  
#             return  
#         printNatural(4-1)  
#         print(4, end=" ")  
  
# def printNatural(3):  
#         if 3 < 1:  
#             return  
#         printNatural(3-1)  
#         print(3, end=" ")  
  
# def printNatural(2):  
#         if 2 < 1:  
#             return  
#         printNatural(2-1)  
#         print(2, end=" ")  
  
# def printNatural(1):  
#         if 1 < 1:  
#             return  
#         printNatural(1-1)  
#         print(1, end=" ")  
  
# def printNatural(0):  
#         if 0 < 1:  
#             return  
#         printNatural(0-1)  
#         print(0, end=" ")
```

```
In [ ]: ▶ #Program to demonstrate double recursion  
def printf(n):  
    #base case  
    if n < 1:  
        return  
    printf(n-1) # head recursion  
    print(n, end=" ")  
    printf(n-1) # tail recursion  
  
# main function  
n = 3  
printf(n)
```

```
In [ ]: ▶ list1 = [1,2]
print(list1.__sizeof__())
list1.append(3)
print(list1.__sizeof__())
list1.append(4)
print(list1.__sizeof__())
list1.append(5)
print(list1.__sizeof__())
list1.append(5)
print(list1.__sizeof__())
list1.append(6)
print(list1.__sizeof__())
list1.append(7)
print(list1.__sizeof__())
list1.append(8)
print(list1.__sizeof__())
list1.append(9)
print(list1.__sizeof__())
list1.pop()
print(list1.__sizeof__())
```

```

In [ ]:  class Node:
        def __init__(self, value):
            self.data = value
            self.next = None

        class LinkedList:
            def __init__(self):
                self.root = None
            def addFirst(self, value):
                newNode = Node(value)
                newNode.next = self.root
                self.root = newNode
            def addLast(self, value):
                newNode = Node(value)
                if self.root == None:
                    self.root = newNode
                else:
                    temp = self.root
                    while temp.next != None:
                        temp = temp.next # move the pointer to the next node
                    temp.next = newNode

            def deleteFirst(self):
                # checking whether the list empty or not
                if self.root != None:
                    # list is not empty
                    self.root = self.root.next

            def deleteLast(self):
                pass

            def printList(self):
                if self.root == None:
                    print("[]")
                else:
                    temp = self.root;
                    print("[", end="")
                    while temp.next != None:
                        print('"' + temp.data + '"', end=", ", sep="")
                        temp = temp.next
                    print('"' + temp.data + '"', end="]", sep="")

myList = LinkedList()
myList.addFirst("Hari")
myList.addFirst("Raj")
myList.addFirst("Jai")
myList.addLast("Subash")
myList.addFirst("Javid")
myList.printList()
myList.deleteFirst()
myList.printList()

```

```
In [ ]: ▶ #Program to implement coin change problem - greedy approach
amt = int(input()) # amount
n = int(input()) # number of denominations
denoms = list(map(int, input().split()))
denoms.sort(reverse=True)
totalCoins = 0
changeDetails = {}
for coin in denoms:
    if amt >= coin:
        cur = amt // coin
        changeDetails[coin] = cur
        totalCoins += cur
        amt = amt % coin # amt %= coin
    if amt == 0:
        break
if amt == 0:
    print(totalCoins)
    print(changeDetails)
else:
    print(-1)
```

```
In [ ]: ▶ # Day - 3
# Program to implement activity selection problem - greedy
n = int(input())
activities = []
for _ in range(n):
    activities.append(list(map(int, input().split())))
# sorting the activities by its ending time
result = [] # to store participated activities
result.append(activities[0])
activities.sort(key=lambda activity : activity[1])
i = 0
while i < n:
    for j in range(i+1, n):
        if activities[j][0] >= activities[i][1]:
            result.append(activities[j])
            i = j;
    i+=1
print(result)
```

```
In [ ]: ▶ #Program to implement fractional knapsack problem
bagSize = int(input())
n = int(input())
products = []
for _ in range(n):
    profit, weight = list(map(int,input().split()))
    ratio = profit / weight
    products.append([ratio,profit, weight])
products.sort(reverse=True)
maxProfits = 0
for r,p,w in products:
    if bagSize >= w:
        maxProfits += p
        bagSize-=w
    else:
        maxProfits += bagSize * r
        bagSize = 0
print(maxProfits)
```

```
In [ ]: ▶ # profit, weight = list(map(int,input().split()))
# print (profit, weight)
l1 = [1,2,3]
x,y,z = l1
print(x,y,z)
```

```
In [ ]: ▶
```

```
In [ ]: ▶ # Day 4
cnt = 0
def fib(n):
    global cnt
    cnt+=1
    if n <= 1:
        return n
    return fib(n-1) + fib(n-2)

#main function
n = int(input())
print(fib(n))
print(cnt)
```


In [4]:  *#Program to find nth term of fibonacci series - using memoization*

```
cnt = 0
def fib(n, dp):
    global cnt
    cnt+=1
    #base case
    if n <= 1:
        return n
    if dp[n] != -1:
        return dp[n]
    dp[n] = fib(n-1, dp) + fib(n-2, dp)
    return dp[n]

#main function
n = int(input())
dp = [-1] * (n+1)
print(fib(n, dp))
print(cnt)
```

5
5
9

In [7]:  *#Program to find nth term of fibonacci series - using tabular method*

```
cnt = 0
def fib(n):
    global cnt
    cnt+=1
    dp = [-1] * (n+1)
    dp[0], dp[1] = 0, 1
    for i in range(2, n+1):
        dp[i] = dp[i-1] + dp[i-2]
    return dp[n]

#main function
n = int(input())
print(fib(n))
print(cnt)
```

5
5
1

In [18]:  *#Climbing stair case - tabular*

```
def ways(n):
    dp = [-1] * (n+1)
    if n == 0:
        dp[0] = 0
    elif n == 1:
        dp[0], dp[1] = 0,1
    else:
        dp[0], dp[1], dp[2] = 0,1,2
    for i in range(3, n+1):
        dp[i] = dp[i-1] + dp[i-2]
#     print (dp)
    return dp[n]

#main function
n = int(input())
print(ways(n))
```

```
5
[0, 1, 2, 3, 5, 8]
8
```

In [22]:  *#Program to implement coin change problem using DP method*

```
def minimumCoins(amt, coins, n):
    # creating a list with amt+1 elements and initialize all element as am
    dp = [amt+1] * (amt+1)
    dp[0] = 0
    for coin in coins:
        for i in range(coin, amt+1):
            dp[i] = min(dp[i], 1+dp[i-coin])
    return dp[amt]

# main function
amt = int(input())
n = int(input())
denoms = list(map(int, input().split()))
print(minimumCoins(amt, denoms, n))
```

```
11
3
6 4 3
3
```

```
In [2]: ▶ # Day - 5
# Program to find Longest increasing subsequence
def longestIncreasingSeq(seq):
    n = len(seq)
    dp = [1] * n
    for i in range(1,n):
        for j in range(i):
            if seq[i]>seq[j]:
                dp[i] = max(dp[i], 1+dp[j])
    return max(dp)

n = int(input())
arr = list(map(int, input().split()))
print(longestIncreasingSeq(arr))
```

```
10
10 22 9 33 21 50 41 60 80 20
6
```

```
In [8]: ▶ #Program to find Length Longest common substring
def longestCommonSubstring(s1, s2):
    m = len(s1)
    n = len(s2)
    dp = [[0] * (n+1) for _ in range(m+1)]
    for i in range(1,m+1):
        for j in range(1,n+1):
            # checking string1 ith character and string2 jth characters are equal
            if s1[i-1] == s2[j-1]:
                # left up diagonal element + 1 if true
                dp[i][j] = 1+dp[i-1][j-1]
            # returning the highest value from two dimensional dp table
    return max([max(row) for row in dp])

str1, str2 = input().split()
print(longestCommonSubstring(str1, str2))
```

```
hish fish
3
```